hashlock.

2025

Penetration Testing

Orlyn (dApp)

April 2025

info@hashlock.com.au

Hashlock Pty Ltd

Table of Contents

CAUTION

THIS DOCUMENT IS A SECURITY PENETRATION TEST REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE THAT COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR THE USE OF THE CLIENT.

Executive Summary

The Orlyn team partnered with Hashlock to conduct a security Penetration Test of their Orlyn Project code base. Hashlock manually and proactively reviewed the code to ensure the project's team and community that the deployed tools were secure.

Project Context

Orlyn creates anonymous, user-owned connections between people and brands using Pocket Crumbs - a new blockchain method for verifying relationships without sharing data.

Orlyn is a decentralized platform that empowers users to take control of their personal data and be rewarded for sharing it with brands. By leveraging blockchain technology, Orlyn ensures that users' data is stored securely and anonymously, allowing them to engage with brands without compromising their privacy.

Orlyn's mission is to take the world's data back from Big Tech and put it in the hands of everyday people, onboarding hundreds of millions into Web3.

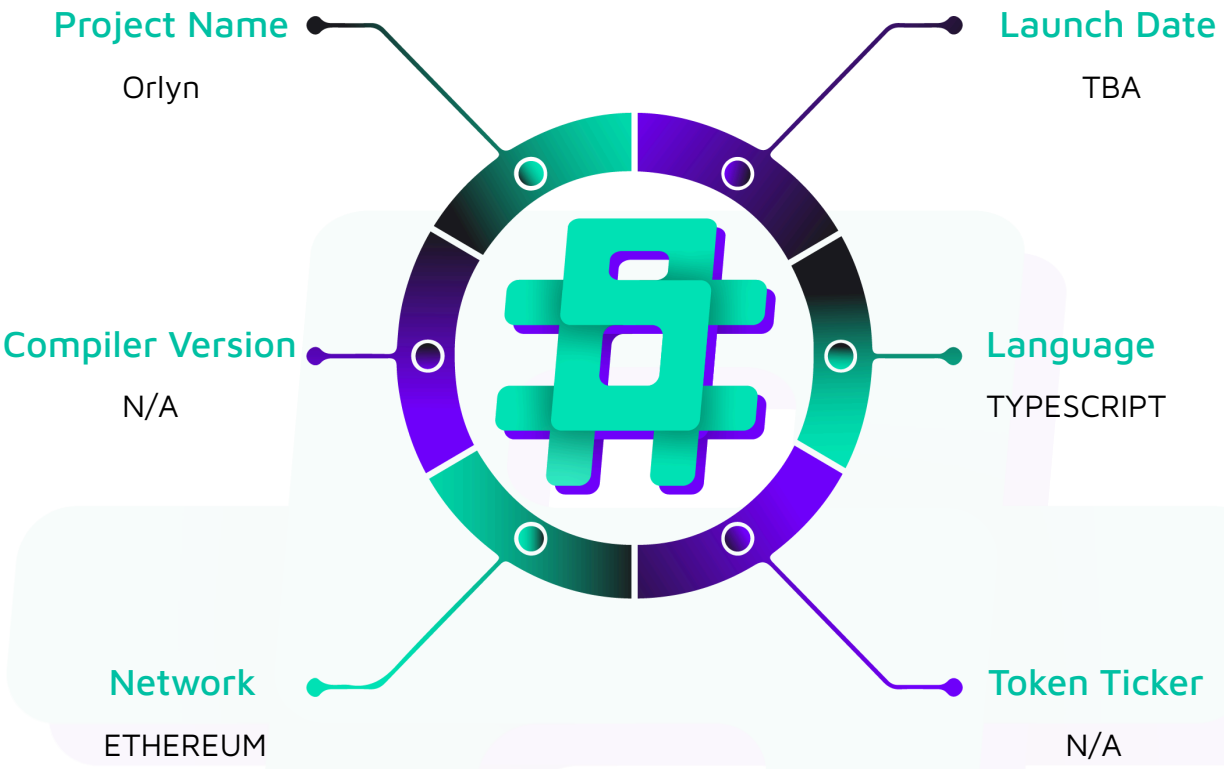
Project Name: Orlyn

Project Type: dApp, Token

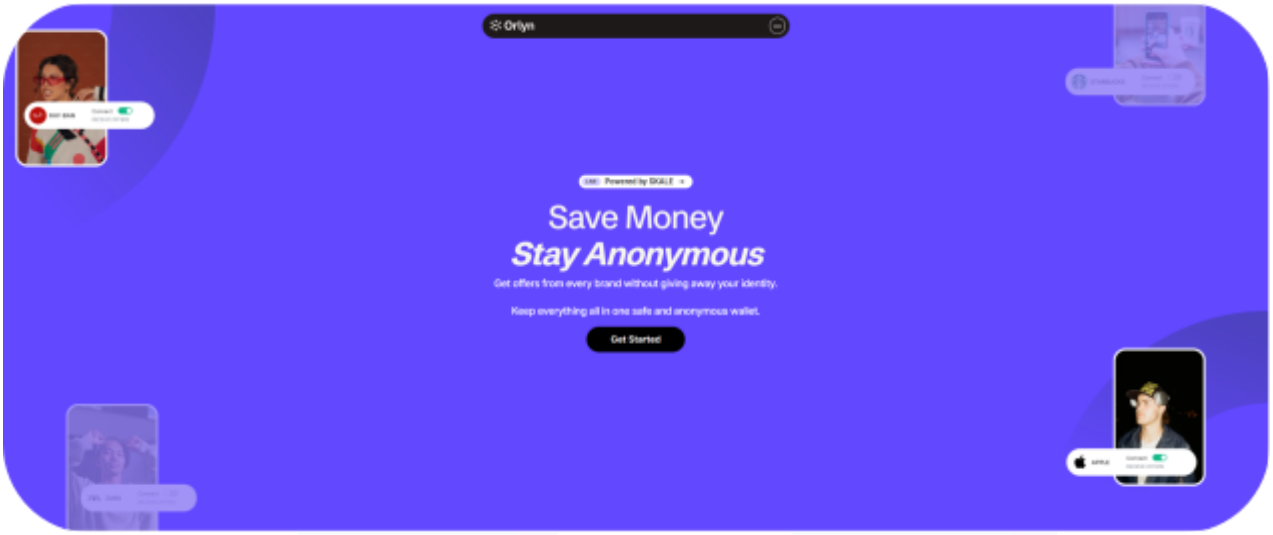
Website: https://www.orlyn.io/

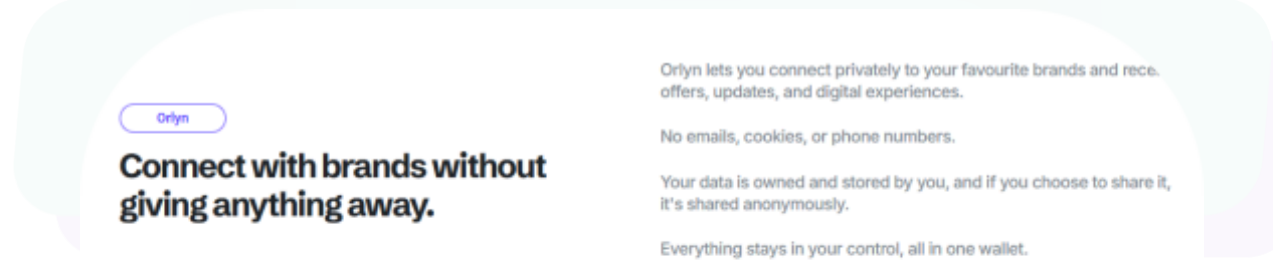
Logo:

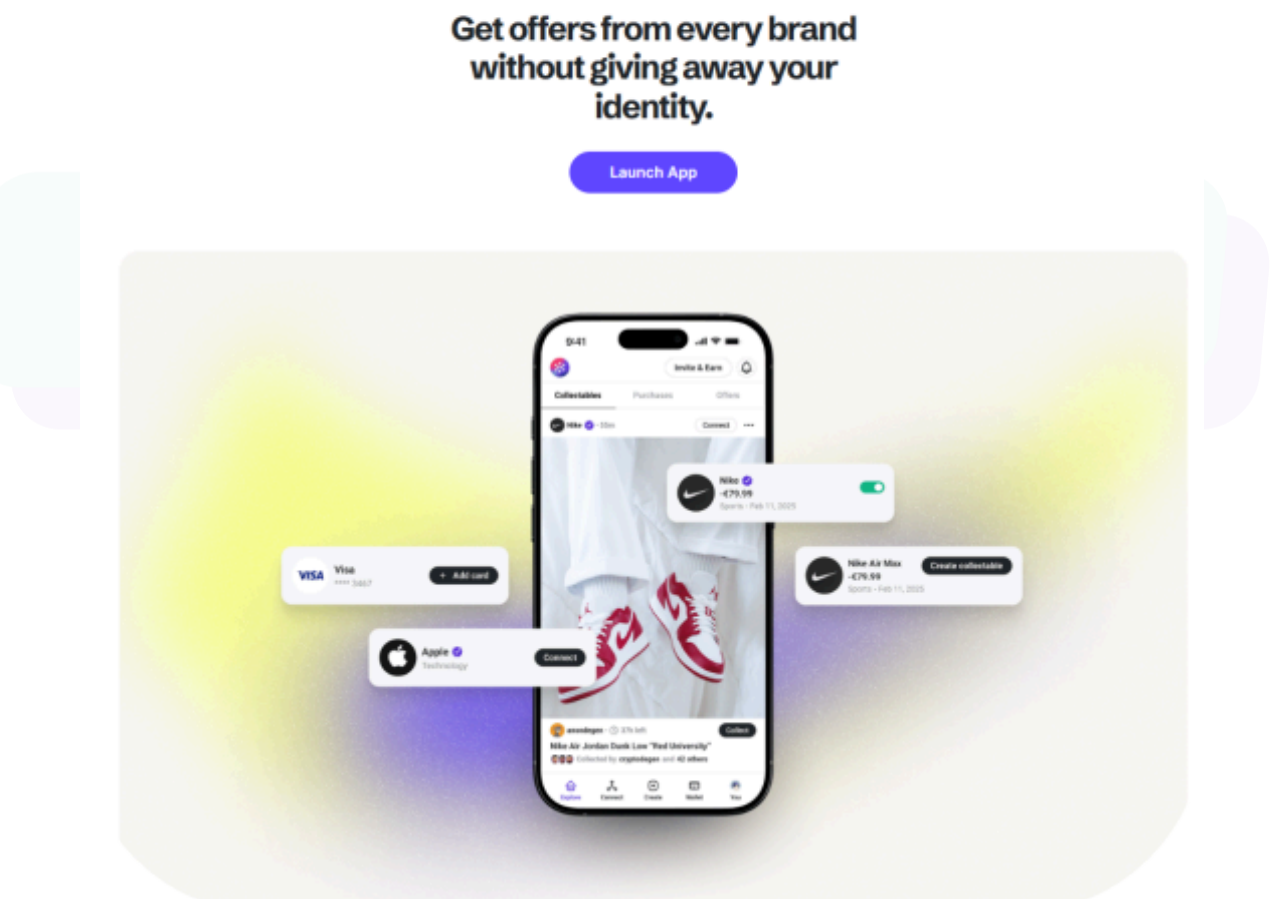
Visualised Context:



Project Visuals:







Penetration Test Scope

At Hashlock, we executed a comprehensive penetration test on the Orlyn project. Our scope included a detailed security assessment, where we rigorously examined the code to identify vulnerabilities and assess overall efficiency. The testing process involved a meticulous manual review, supplemented by advanced software-assisted techniques, to ensure thorough analysis and accuracy.

Description	**Orlyn Project Code Base**
Platform	**Ethereum / Typescript / Web apps & APIs**
Penetration Test Date	**April, 2025**
Repository/Website URL/IP Tested	https://github.com/wal8io/wal8-backend/tree/main/src
Fix Review GitHub Commit Hash (If applicable)	d575f687aacc3c95677603753328deedbb2f76b9

Note: *Hashlock initially audited the code associated with the "Audited GitHub Commit Hash" and the findings presented in this report pertain specifically to that commit. For the "Fix Review GitHub Commit Hash", Hashlock solely verified the status of the identified findings and did not conduct a comprehensive review to assess any additional code implementations.*

Security Rating

After Hashlock's Penetration Test, we found the code base to be **"Secure"**. The code follows simple logic, with correct and detailed ordering.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on-chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Penetration Test Findings section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

2 High-severity vulnerabilities

1 Medium-severity vulnerabilities

4 Low-severity vulnerabilities

1 QA

Caution: *Hashlock's Penetration Tests do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system’s performance or security.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Penetration Test Findings

High

[H-01] API - Authorization weakness in multiple endpoints

Description

Multiple endpoints allow accessing sensitive data across user boundaries by simply modifying user IDs in requests, fundamentally breaking the application's authorization model. This is global for the API and seems that while authentication is implemented, authorization is not, except for the `requireApiKey` function used in some endpoints.

Vulnerability Details

`wal8-backend-main/src/routes/user.ts` allows access to user information for any authenticated user:

```
// GET /user - Get the current user's information
// Only authenticated users can access this route
// returns the user's address, username, and data signing address
userRoutes.get('/user', authMiddleware, async (c) => {
  const userFromDB = await c.get('userFromDB');

  if (!userFromDB) {
    return c.json({ error: 'User not found' }, 404);
  }

  return c.json({
    user: {
      address: userFromDB.walletAddress,
```

```
      username: userFromDB.username,
      dataSigningAddress: userFromDB.dataSigningAddress,
      pushNotificationToken: userFromDB.pushNotificationToken,
      plaidToken: userFromDB.plaidToken,
      profilePicture: userFromDB.profilePicture,
      userId: userFromDB.id
    }
  });
});
```

Other occurrences are global to the API, for instance, purchase records can be accessed by any authenticated user in `wal8-backend-main/src/routes/purchaseRecords.ts`:

```
purchaseRecordsRoutes.get('/:userId',
  authMiddleware,
  authedAndOnboarded,
  async (c) => {
    const { userId } = c.req.valid('param');
    // No verification that userId matches authenticated user
     const records = await PurchaseRecordService.getLastPurchaseRecordsForUser(parseInt(userId), 10);
    return c.json({ records });
  }
);
```

Similar issues exist in ticket access (which is not part of the documented API routes) in `wal8-backend-main/src/routes/tickets.ts`

```
ticketsRoutes.get('/:id',
  authMiddleware,
```

```
    authedAndOnboarded,
  async (c) => {
    // No verification that ticket belongs to authenticated user
    const ticket = await TicketsService.getTicketById(id);
 return c.json({ success: true, data: ticket });
  }
);
```

Impact

Authenticated users can access sensitive private information of any user in the system, completely breaking confidentiality guarantees.

Recommendation

Implement user ownership verification in all data access endpoints, comparing the requested resource owner with the authenticated user's ID.

Status

Resolved

[H-02] API - Undocumented API endpoints create an extensive attack surface

Description

The application implements numerous API endpoints not documented in the original `README`, creating a shadow API attack surface that may escape security testing and monitoring.

For instance, with a quick review, it was identified that image URLs are not validated possibly allowing for linking malicious content or scripts, notifications can be created by unsubscribed users, or aforementioned tickets can be viewed by non-owners.

Vulnerability Details

The following endpoints in `wal8-backend-main/src/routes/index.ts` are implemented but not properly documented in `README`:

```
apiRoutes.route('/plaid', plaidRoutes);

apiRoutes.route('/purchase-records', purchaseRecordsRoutes);

apiRoutes.route('/brands', brandsRoutes);

apiRoutes.route('/cred8', cred8Routes);

apiRoutes.route('/tickets', ticketsRoutes);
```

Most of them also have excessive sub-endpoints with multiple functionalities.

Recommendation

Update documentation to include all implemented endpoints and align endpoint naming between documentation and implementation. Implement API gateway controls that reject requests to undocumented endpoints.

If endpoints are not to be used, do not register an API route for them.

Status

Resolved

Medium

[M-01] API - Lack of rate limiting

Description

API endpoints lack any form of rate limiting, enabling attackers to flood the system with requests that could overwhelm database connections and CPU resources.

Vulnerability Details

For instance, authentication endpoints have no protection against excessive requests, which may be used to force the underlying server to perform an excessive number of operations:

```
authRoutes.post('/nonce', zValidator('json', nonceSchema), async (c) => {
  const { address } = await c.req.json();
  // No rate limiting check
  const nonce = siweService.generateNonce();
  const expires = await siweService.storeNonce(address, nonce);
  return c.json({ nonce, expires });
});
authRoutes.post('/verify', zValidator('json', verifySchema), async (c) => {
  // No rate limiting check
  const { message, signature } = await c.req.json();
  const result = await siweService.verifySiweMessage(message, signature);
  // ...
});
```

Impact

Attackers can overwhelm the system with authentication requests, causing database connection exhaustion, increased server load, and potential denial of service.

Recommendation

Implement rate-limiting middleware for authentication endpoints based on IP address and wallet address, with appropriate limits based on expected legitimate traffic patterns. A WAF such as Cloudflare might be helpful with production deployment.

Status

Acknowledged

Low

[L-01] API - Missing HTTP security headers

Description

The API server lacks essential security headers and implements a wildcard `CORS` policy that, while partially mitigated by `JWT` authentication, still represents a deviation from security best practices.

The application sets a permissive `CORS` configuration:

```
app.use('*', cors({
  origin: '*',
  allowMethods: ['GET', 'POST'],
  allowHeaders: ['Content-Type', 'Authorization'],
  exposeHeaders: ['Content-Length', 'X-Request-Id'],
  maxAge: 86400,
}));
```

Additionally, no standard security headers are implemented:

```
Strict-Transport-Security
X-Content-Type-Options
X-Frame-Options
Content-Security-Policy
```

While the use of `JWT` authentication reduces the risk of `CSRF` attacks despite permissive `CORS`, the application remains more vulnerable to clickjacking, `MIME-type` confusion, and other browser-based attacks when directly accessed. Information leakage about API structure remains possible.

Recommendation

Implement security headers through middleware and restrict `CORS` to specific origins

```
app.use('*', (c, next) => {
  c.header('Strict-Transport-Security', 'max-age=31536000; includeSubDomains');
  c.header('X-Content-Type-Options', 'nosniff');
  c.header('X-Frame-Options', 'DENY');
  return next();
});

// Replace wildcard CORS with specific origins
app.use('*', cors({
  origin: ['https://app.example.com', 'https://admin.example.com'],
  // other settings remain the same
}));
```

Status

Resolved

[L-02] API - Potential SQL Injection

Description

In `purchaseRecordService.ts`, there is an usage of `sql.raw()` with unvalidated input in partition creation that can allow `SQL` injection if `year` ever comes from an untrusted source. This is only endpoint that seem not to benefit from Drizzle framework and use a raw `SQL` query. An attacker who can influence `year` could inject arbitrary `SQL`, leading to data leakage or destruction.

```
await db.execute(sql`
  CREATE TABLE IF NOT EXISTS purchase_records_${sql.raw(year.toString())}
  PARTITION OF purchase_records
  FOR VALUES FROM (${sql.raw(`'${year}-01-01'`)})
  TO (${sql.raw(`'${nextYear}-01-01'`)})
`);
```

Recommendation

Validate/whitelist `year` as an integer and use parameterized queries rather than `sql.raw()`.

Status

Resolved

[L-03] API - Verbose Error Messages

Description

Detailed internal errors (stack traces, parse errors) are returned to clients, leaking implementation details. This issue occurs globally in the API, for example:

```
// routes/auth.js

return c.json({ error: 'Failed to parse SIWE message: Invalid format' }, 400);
```

Such information leakage aids attackers in crafting targeted requests.

Recommendation

Catch errors globally and return sanitized messages (e.g. "Invalid request"). Log full details server-side only.

Status

Acknowledged

[L-04] API - Vulnerable dependencies in use

Description

Security scans using `npm audit` and `snyk` identified several vulnerabilities in project dependencies, including critical issues in cryptographic libraries and high severity DoS vulnerabilities.

While this does not mean that the project is directly vulnerable, as for exploitation certain vulnerable functions have to be used and accessible for user input, it is a best practice to resolve all known vulnerabilities. Below list contains output of npm audit:

```
# npm audit report

cookie  <0.7.0
cookie accepts cookie name, path, and domain with out of bounds characters -
https://github.com/advisories/GHSA-pxg6-pf52-xh8x
fix available via `npm audit fix`
node_modules/cookie
  youch  2.0.4 - 3.3.3
  Depends on vulnerable versions of cookie
  node_modules/youch
    miniflare  <=0.0.0-fff677e35 || 1.0.0 - 1.4.1 || 3.20250204.0 - 3.20250310.2 ||
4.20250214.0-rc.0 - 4.20250321.1
    Depends on vulnerable versions of youch
    node_modules/miniflare
      wrangler  <=4.10.0
      Depends on vulnerable versions of esbuild
      Depends on vulnerable versions of miniflare
      node_modules/wrangler

elliptic  <=6.6.0
Severity: critical
Elliptic's EDDSA missing signature length check -
https://github.com/advisories/GHSA-f7q4-pwc6-w24p
Elliptic's ECDSA missing check for whether leading bit of r and s is zero -
https://github.com/advisories/GHSA-977x-g7h5-7qgw
Elliptic allows BER-encoded signatures -
```

```
https://github.com/advisories/GHSA-49q7-c7j4-3p7m
Valid ECDSA signatures erroneously rejected in Elliptic -
https://github.com/advisories/GHSA-fc9h-whq2-v747
Elliptic's verify function omits uniqueness validation -
https://github.com/advisories/GHSA-434g-2637-qmqr
Elliptic's private key extraction in ECDSA upon signing a malformed input (e.g. a
string) - https://github.com/advisories/GHSA-vjh7-7g9h-fjfh
fix available via `npm audit fix`
node_modules/ethers/node_modules/elliptic
  @ethersproject/signing-key  <=5.7.0
  Depends on vulnerable versions of elliptic
  node_modules/ethers/node_modules/@ethersproject/signing-key

esbuild  <=0.24.2
Severity: moderate
esbuild enables any website to send any requests to the development server and read the
response - https://github.com/advisories/GHSA-67mh-4wv8-2f99
fix available via `npm audit fix --force`
Will install @vercel/node@2.3.0, which is a breaking change
node_modules/@esbuild-kit/core-utils/node_modules/esbuild
node_modules/drizzle-kit/node_modules/esbuild
node_modules/esbuild
node_modules/wrangler/node_modules/esbuild
  @esbuild-kit/core-utils  *
  Depends on vulnerable versions of esbuild
  node_modules/@esbuild-kit/core-utils
    @esbuild-kit/esm-loader  *
    Depends on vulnerable versions of @esbuild-kit/core-utils
    node_modules/@esbuild-kit/esm-loader
      drizzle-kit  0.9.1 - 0.9.54 || >=0.12.9
      Depends on vulnerable versions of @esbuild-kit/esm-loader
      Depends on vulnerable versions of esbuild
      node_modules/drizzle-kit
  @vercel/node  >=2.3.1
  Depends on vulnerable versions of esbuild
  Depends on vulnerable versions of path-to-regexp
  Depends on vulnerable versions of undici
  node_modules/@vercel/node
```

```
path-to-regexp  4.0.0 - 6.2.2
Severity: high
path-to-regexp outputs backtracking regular expressions -
https://github.com/advisories/GHSA-9wv6-86v2-598j
fix available via `npm audit fix --force`
Will install @vercel/node@2.3.0, which is a breaking change
node_modules/path-to-regexp

undici  4.5.0 - 5.28.4
Severity: moderate
Use of Insufficiently Random Values in undici -
https://github.com/advisories/GHSA-c76h-2ccp-4975
fix available via `npm audit fix --force`
Will install @vercel/node@2.3.0, which is a breaking change
node_modules/undici

vite  6.2.0 - 6.2.5
Severity: moderate
Vite bypasses server.fs.deny when using ?raw?? -
https://github.com/advisories/GHSA-x574-m823-4x7w
Vite has a `server.fs.deny` bypassed for `inline` and `raw` with `?import` query -
https://github.com/advisories/GHSA-4r4m-qw57-chr8
Vite allows server.fs.deny to be bypassed with .svg or relative paths -
https://github.com/advisories/GHSA-xcj6-pq6g-qj4x
Vite has an `server.fs.deny` bypass with an invalid `request-target` -
https://github.com/advisories/GHSA-356w-63v5-8wf4
fix available via `npm audit fix`
node_modules/vite

ws  7.0.0 - 7.5.9
Severity: high
ws affected by a DoS when handling a request with many HTTP headers -
https://github.com/advisories/GHSA-3h5v-q93c-6h6q
fix available via `npm audit fix`
node_modules/ethers/node_modules/ws
  @ethersproject/providers  <=5.7.2
  Depends on vulnerable versions of ws
```

```
  node_modules/ethers/node_modules/@ethersproject/providers
    ethers  5.0.15 - 5.7.2
    Depends on vulnerable versions of @ethersproject/providers
    Depends on vulnerable versions of @ethersproject/signing-key
    node_modules/ethers

17 vulnerabilities (4 low, 7 moderate, 5 high, 1 critical)
```

The list below contains all issues identified with snyk:

```
Issues to fix by upgrading:


  Upgrade ethers@5.7.2 to ethers@6.0.0 to fix
  ✗ Improper Verification of Cryptographic Signature [High
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-8172694] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4
  ✗ Denial of Service (DoS) [High
Severity][https://security.snyk.io/vuln/SNYK-JS-WS-7266574] in ws@7.4.6
    introduced by ethers@5.7.2 > @ethersproject/providers@5.7.2 > ws@7.4.6
  ✗ Improper Verification of Cryptographic Signature [Critical
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-8187303] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4 and
65 other path(s)
  ✗ Improper Verification of Cryptographic Signature [Critical
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-7577916] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4
  ✗ Improper Verification of Cryptographic Signature [Critical
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-7577917] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4
  ✗ Improper Verification of Cryptographic Signature [Critical
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-7577918] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4
  ✗ Information Exposure [Critical
Severity][https://security.snyk.io/vuln/SNYK-JS-ELLIPTIC-8720086] in elliptic@6.5.4
    introduced by ethers@5.7.2 > @ethersproject/signing-key@5.7.0 > elliptic@6.5.4


Issues with no direct upgrade or patch:
  ✗ Prototype Pollution [High
Severity][https://security.snyk.io/vuln/SNYK-JS-WEB3UTILS-6229337] in web3-utils@1.10.4
    introduced by @skaleproject/pow-ethers@0.3.3 > @skaleproject/pow@0.3.2 >
web3-utils@1.10.4
  This issue was fixed in versions: 4.2.1
```

Recommendation

Update dependencies to secure versions.

Status

Resolved

QA

[Q-01] package.json - Dependencies are not pinned to exact versions

Description

The application uses external dependencies, some of which are not pinned to an exact version but set to a compatible version (`^x.x.x`). This can potentially allow for dependency attacks. Similar attacks were seen in the past, for example, Copay Bitcoin Wallet was attacked in such a way.

Recommendation

Pin all external dependencies to exact versions (`x.x.x`) instead of using compatible version ranges (`^x.x.x`) to mitigate the risk of dependency-based attacks.

Status

Resolved

Conclusion

After Hashlock's analysis, the Orlyn project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our Penetration Tests a collaborative effort. The objective of our security Penetration Tests is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security Penetration Test process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future Penetration Tests. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the code base under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other Penetration Test results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we still need to verify the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown not to represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed the code bases in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the code base source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the Penetration Test makes no statements or warranties on the security of the code. It also cannot be considered a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this code base.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Code is deployed and executed on a platform. The platform, its programming language, and other software related to the code base can have their own vulnerabilities that can lead to attacks. Thus, the Penetration Test can't guarantee the explicit security of the Penetration Tested code bases.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au